

“逐鹿”Alpha 专题报告（二十二）——Factor Zoo II 基本面因子挖掘统一框架

姚紫薇 王超 中信建投金工及基金研究团队 2024年08月09日 08:31 上海

【重要提示】：通过本订阅号发布的观点和信息仅供中信建投证券股份有限公司（下称“中信建投”）客户中符合《证券期货投资者适当性管理办法》规定的机构类专业投资者参考。因本订阅号暂时无法设置访问限制，若您并非中信建投客户中的机构类专业投资者，为控制投资风险，请您取消关注，请勿订阅、接收或使用本订阅号中的任何信息。对由此给您造成的不便表示诚挚歉意，感谢您的理解与配合！

核心观点.摘要

本文我们将之前系列基本面因子挖掘的算法进行了统一的整合及优化，构建出了基本面因子挖掘统一框架，相比于传统的基本面因子挖掘方法，我们提出的框架能够适用于各类主观及自动化因子挖掘方案，并且对于因子的量纲及频率进行了自动化处理，确保了生成因子的合法性和有效性，对因子的适应度也进行了改进，加入了相关性的惩罚项，能够生成更加多样的基本面因子。

主要内容

简介

在先前的报告系列中，我们已经探究了众多因子挖掘的技术与方法。这些包括但不限于基于枚举法的OPENFE，以启发式算法为核心的AlphaZero，以及将启发式与枚举法相结合、并融合领域知识的因子挖掘算法，皆适用于多种类型因子挖掘的具体场景。本文我们将之前系列基本面因子挖掘的算法进行了统一的整合及优化，构建出了基本面因子挖掘统一框架。

框架

在量化投资领域，基本面因子的分析面临诸多挑战：数据格式的多样性、更新频率的差异、缺失值的普遍性、数据维度的复杂性、因子间的高相关性，以及数据发布与公告日期的不一致性等。针对这些问题，本文提出了一种创新的统一基本面因子框架。该框架包括了因子生成，因子计算，因子验证，因子进化，因子筛选等模块，对每个模块都进行了优化，使其能够更加高效合理的挖掘基本面因子。该框架不仅整合了多种因子生成技术，还能够有效处理不同频率和量纲的数据融合问题。通过这一方法，我们能够提炼出适应性强、低相关性的优质基本面因子。

结论

在本篇研究中，我们在先前系列因子挖掘工作的基础上，精心构建了一个全面的基本面因子挖掘统一框架，该框架不仅整合了多种因子生成技术，并且能够自动优化基本面因子的频率和量纲。在此框架中融入了一系列常用算子，包括元素算子、时序算子和截面算子等，此外，为了进一步提升计算效率，我们在框架的底层采用了cython与流式计算技术的结合，这显著提高了因子计算的速度和性能。在因子筛选阶段，我们特别引入了因子相关性惩罚机制。这一机制的运用有效促进了因子的多样性，确保了最终生成的因子集合在保持低相关性的同时，也能捕捉到市场的有效信息。

本文是Factor Zoo系列报告的第二篇。Factor Zoo旨在深入挖掘并剖析各类Alpha因子的表现特征，继承并扩展了我们先前Model Zoo系列的研究范畴。在这两大系列的研讨中，我们致力于对广泛的模型

与因子进行细致的考察与大规模的检验，力求提炼出宝贵的经验规律，从而为未来因子与模型的创新发展奠定坚实的基础。

在先前的报告系列中，我们已经探究了众多因子挖掘的技术与方法。这些包括但不限于基于枚举法的OPENFE，以启发式算法为核心的AlphaZero，以及将启发式与枚举法相结合、并融合领域知识的因子挖掘算法，皆适用于多种类型因子挖掘的具体场景。

本文我们将之前系列基本面因子挖掘的算法进行了统一的整合及优化，构建出了基本面因子挖掘统一框架，相比于传统的基本面因子挖掘方法，我们提出的框架能够适用于各类主观及自动化因子挖掘方案，并且对于因子的量纲及频率进行了自动化处理，确保了生成因子的合法性和有效性，对因子的适应度也进行了改进，加入了相关性的惩罚项，能够生成更加多样的基本面因子。

2 相关工作

在本篇研究中，我们基于先前发布的逐鹿因子挖掘系列报告进行了全面的整合与进一步的优化。在深入探讨本文的研究内容之前，让我们先简要回顾一下我们在之前系列报告中所开展的工作。

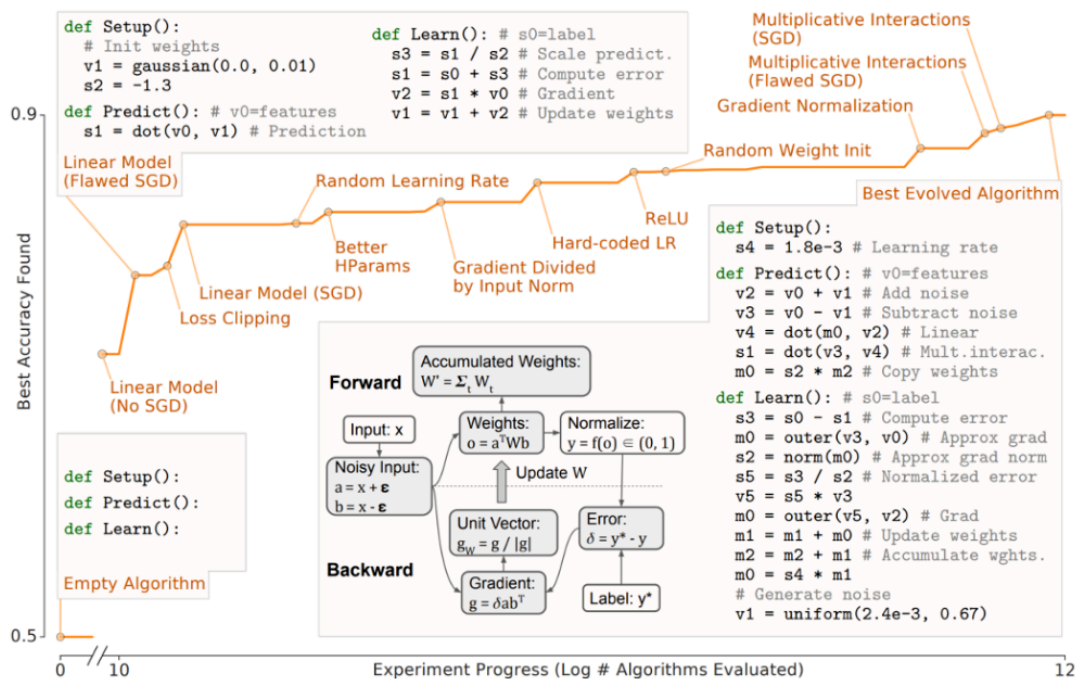
2.1 AlphaZero

AlphaZero是我们将google的AutoML-Zero模型应用于因子挖掘领域，结合实际情况对模型做了相应的改进，构建的因子挖掘框架。

AutoML-Zero是基于程序结构的个体结合正则化进化算法的方式，将程序内的代码不断变异进化，最终生成目标程序。在原论文中，作者通过在CIFAR-10的数据集上训练，发现算法能够不断进化，从线性模型，到损失函数，梯度下降，激活函数等等，实现了完整的MLP过程。

在AutoML-Zero的基础上，我们进一步发展并构建了AlphaZero框架。与传统的遗传编程以及AutoML-Zero相比，AlphaZero在提升因子挖掘效率和增强因子可解释性方面做出了显著的优化和改进。首先，我们对所有数据采用了量纲化处理，避免了在因子挖掘中经常出现的不同量纲之间的因子运算，并且要求最终生成的因子为无量纲因子，这样使得因子可解释性问题有所缓解。其次，我们对合成因子的长度进行了严格控制，以避免因子计算过于复杂，从而减少过拟合的风险。最后，我们引入了热启动机制、退化变异以及灾难算法等策略。这些策略不仅提高了算法的进化效率，而且在正则化的基础上增加了物种的多样性，促进了因子的分散度，从而提高了结果的鲁棒性和创新性。

图 1: AutoML-Zero 算法



数据来源: AutoML-Zero: Evolving Machine Learning Algorithms From Scratch, 中信建投

2.2 OpenFE

OpenFE是一个基于枚举法的Expand-And-Reduce框架，首先通过基础特征以及算子的排列组合构建具有一定结构的风格因子。而后通过两步的筛选步骤，对因子进行筛选，保留最终特征重要性最高的因子。

图 2: OpenFE 结构

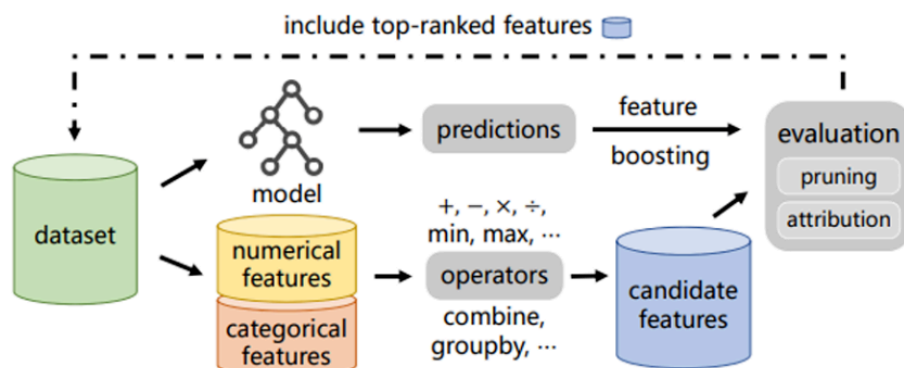


Figure 1: The overview of OpenFE.

数据来源: OpenFE: Automated Feature Generation with Expert-level Performance, 中信建投

在构造因子时，我们借鉴已有的成熟因子结构，例如ROE,PE等，将其拓展为二阶因子结构，并且限制分子分母的数据来源，确保其具有清晰的金融学含义，采用枚举法的方法生成所有可能的因子形式，生成了共计约70万因子。

在处理大量生成的因子时，传统因子检验方法的效率往往不尽人意。为了提高效率，OpenFE采用了一种两阶段的检验流程：

第一步：应用连续二分法进行初步筛选。在面对大量因子时，该方法首先使用少量数据进行快速筛选，以减少因子的数量。随着筛选的进行，逐步增加用于检验的数据量，并重复这一过程。这一步骤将持续进行，直到因子数量降至符合特定要求的水平，从而形成一个单因子效果显著的因子集合。

第二步：进行多因子检验。将第一步筛选出的因子集合输入模型进行全面训练，以深入分析每个因子的有效性，并且剔除了因子相关性的影响。最终识别并提炼出最终表现优异的因子列表

表 1:风格因子结构

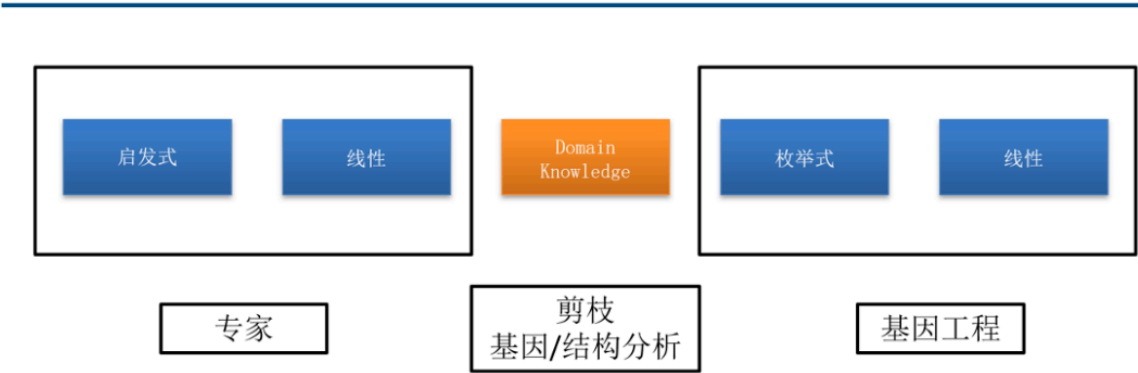
风格	因子结构	说明
杠杆因子	$CSRANK((a \pm b)/(c \pm d))$	$a,b,c,d \in$ 资产负债表
收益因子	$CSRANK((a \pm b)/(c \pm d))$	$a,b \in$ 损益表 $c,d \in$ 资产负债表
质量因子	$CSRANK((a \pm b)/(c \pm d))$	$a,b \in$ 损益表 $c,d \in$ 现金流量表
估值因子	$CSRANK((a \pm b)/market_value)$	$a,b \in$ 损益表/现金流量表/资产负债表
成长因子	$CSRANK(x*f(a) \pm y*f(b))$	$a,b \in$ 损益表/现金流量表/资产负债表, $f \in$ QOQ/YOY, $x,y \in [0,5]$

资料来源：中信建投证券

2.3 领域知识

领域知识是将进化算法作为专家，通过自动化因子挖掘的方式生成优秀的因子种群，对优秀种群做进一步的分析，包括剪枝，基因结构分析等，从种群中提取占比最高的基因以及结构，这些基因是构建优秀因子组合的基础。利用这些精选的基因和结构，我们可以通过人工合成的方式创造新的因子组合，这些新合成的因子将经过严格的检验和筛选流程，以确保它们的有效性。

图 3:领域知识因子挖掘框架



数据来源：中信建投证券

3 框架

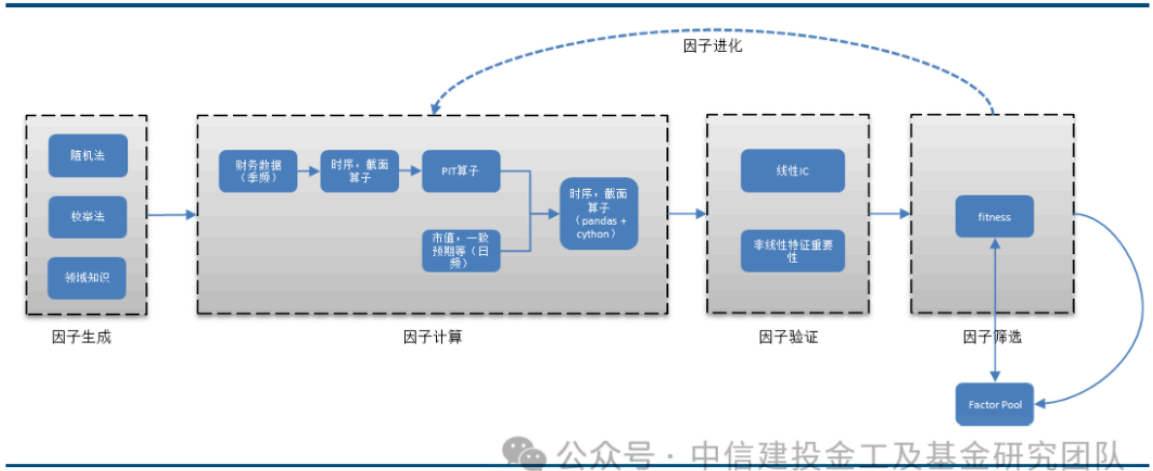
结合之前的研究工作，本文提出了基本面因子挖掘的统一框架，旨在解决基本面因子挖掘中存在的诸多问题。

在量化投资领域，基本面因子的分析面临诸多挑战：数据格式的多样性、更新频率的差异、缺失值的普遍性、数据维度的复杂性、因子间的高相关性，以及数据发布与公告日期的不一致性等。

传统在构建基本面因子时，通常是根据发布日期将因子转换为固定频率，或者在所有公告发布完成后在固定日期构建因子，这两种方法限制了基本面因子挖掘方法的多样性和有效性。

针对这些问题，本文提出了一种创新的统一基本面因子框架。该框架包括了因子生成，因子计算，因子验证，因子进化，因子筛选等模块，对每个模块都进行了优化，使其能够更加高效合理的挖掘基本面因子。该框架不仅整合了多种因子生成技术，还能够有效处理不同频率和量纲的数据融合问题。通过这一方法，我们能够提炼出适应性强、低相关性的优质基本面因子。

图 4:因子挖掘框架



数据来源：中信建投证券

3.1 因子生成

在生成基本面因子时，我们采用了以下三种策略：

随机法：这种方法通过随机方式构造因子结构，主要用于启发式算法中的种群初始化。在生成个体时，我们会限制其结构的复杂度以及量纲的合法性，以确保因子具有简洁性并保持较高的可解释性。

枚举法：与之前openfe算法生成因子一致，在因子生成过程中通过设定因子结构的约束条件，如 $(a+b/c+d)$ ，在给定的子空间内进行全局搜索，以期找到局部最优解。

领域知识：这种方法依赖于专家的主观经验，通过专业知识构造一系列基本面因子。然后利用启发式算法不断优化这些因子，结合剪枝和基因结构分析等技术，筛选出适应度较高的基因及个体。接着，通过枚举法合成新的因子个体，并从中挑选出表现优异的个体。

3.2 因子计算

在本框架中，因子计算是其核心功能。正如之前讨论的，基本面数据面临多种挑战。为了灵活应对这些问题，我们构建了一个双层的因子计算结构。具体来讲，对于每一个数据，我们会设置其量纲以及频率属性，在进行因子计算时，会自动根据其量纲以及频率进行处理，确保运算的合法性。

第一层 主要负责原始财务数据的处理。这包括资产负债表、现金流量表、利润表以及各种财务指标。为了确保数据在时间序列上的可比性，我们分别将现金流量表和利润表等时间段数据统一转换为单季度数据。这一层不仅能够处理截面数据，更重要的是能够处理财务数据的时序数据，例如滚动年度（TTM）、同比（YOY）、环比（QOQ）算子等。在计算这些时序数据时，我们会按报告期的顺序对数据进行排序和处理。

第二层 则专注于不同频率因子的结合。例如，在计算市盈率（PE）因子时，分子端的“E”代表季度财务数据，而分母端的“P”代表日频数据。在处理不同频率的数据时，我们会首先对低频数据按照信息发布日期进行隐式的频率转换（PIT），将其转换为高频数据，然后再与其他高频数据进行计算。

在进行数据计算时，还需注意数据的量纲差异。例如，营收、利润等财务数据具有货币单位的量纲，而一些财务指标如净资产收益率（ROE）、净利润同比增长率则没有量纲。具有相同量纲的数据可以进行常规的数学运算，如加减除等，根据规则得到相应量纲的结果。对于不同量纲的数据，只能进行特定的运算，例如货币单位的数据与无量纲的数据之间只能进行乘法运算，结果仍为货币单位的数据。

尽管市值也具有货币单位的量纲，但将其与营收等货币单位的数据直接进行加法或减法运算显然是不合适的。因此，我们会为这两类数据设置不同的量纲，并仅允许它们之间进行除法运算。这种处理方式确保了计算的准确性和合理性。

在构建算子时，除了常规的运算定义，同样会设置算子输入和输出的量纲要求，具体的定义方式如下表所示，通过这些算子定义，能够避免构造出不符合逻辑的基本面因子。

表 2:算子集合

算子	解释	量纲要求
元素运算符		
add(m1, m2)	$m1+m2$	输入相同量纲, 输出不改变量纲
sub(m1, m2)	$m1-m2$	输入相同量纲, 输出不改变量纲
div(m1, m2)	$m1/m2$	输入相同量纲, 输出无量纲
mul(m1, m2)	$m1*m2$	输入为不同量纲, 输出为带量纲
时间序列运算符		
ts_mean(m,v)	过去 v 期 m 的平均值	输出不改变量纲
ts_std(m,v)	过去 v 期 m 的标准差	输出不改变量纲
ts_delay(m,v)	m 滞后 v 期	输出不改变量纲
ts_delta(m,v)	m 与过去 v 期的差值	输出不改变量纲
ts_pct(m,v)	m 与过去 v 期的变化率	输出无量纲
ts_max(m,v)	过去 v 期 m 的最大值	输出不改变量纲
ts_min(m,v)	过去 v 期 m 的最小值	输出不改变量纲
ts_min_max_diff(m,v)	过去 v 期 m 的最大值与最小值的差	输出不改变量纲
yoy (m)	m 的同比值	输出无量纲
qoq(m)	m 的环比值	输出无量纲
ttm(m)	m 的 TTM 值	输出不改变量纲
ts_slope (m,v)	过去 v 期 m 的斜率	输出不改变量纲
ts_resi (m,v)	过去 v 期 m 的残差	输出不改变量纲
ts_rsquare (m,v)	过去 v 期 m 的 r2	输出无量纲
ts_regression_slope (m1,m2,v)	过去 v 期 m1 对 m2 回归的斜率	输入相同量纲, 输出无量纲
ts_regression_resi(m1,m2,v)	过去 v 期 m1 对 m2 回归的残差	输入相同量纲, 输出不改变量纲
ts_regression_rsquare (m1,m2,v)	过去 v 期 m1 对 m2 回归的 r2	输入相同量纲, 输出无量纲
横截面运算符		
cs_norm(m)	m 的横截面标准化	输出无量纲
cs_rank(m)	m 的横截面排序	输出无量纲

资料来源: 中信建投

其中m代表底层数据, 包括三大财务报表, 财务指标, 一致预期, 市值等, 对于这些原始数据, 我们会为其定义频率和量纲属性。在进行数据分析和运算时, 将依据既定的规则对这些属性进行自动处理。v代表常数项, 取值为1, 2, 4, 8, 12, 对于季频因子, 常数项单位为季度, 取值从一个季度到三年, 如果为日频因子, 单位为月, 时间长度从一个月到一年。

在进行不同频率数据的结合时, 需要不断在发布日期和公告日期之间排序, 以及不同日期之间的合并, 利用pandas能够灵活的实现这些操作, 而对于底层复杂运算, 尤其是时序滚动算子, 采用一般的groupby+rolling apply的计算复杂度为 $O(M*T*N)$, M为股票数量, T为时间长度, N正比于Rolling的长度, 计算相率非常慢。为提升计算性能, 我们特别针对一些较为复杂的算子, 如斜率(slope)、残差(resi)、rsquare等, 采取了与Factor Zoo I相似的底层优化策略。通过结合Cython与流式计算技术, 将计算复杂度降为 $O(B*T)$, 能够显著增强计算速度, 从而实现更高效的数据处理。

以ts_regression_slope算子为例, 如果使用传统的scipy.stats.linregress函数, 对一只股票两个因子间进行1000次T为12的滚动回归, 耗时约8.15秒, 采用cython+流式计算技术, 耗时约

0.07秒，效率提升116倍。其中cython带来的提升约为9.6倍，流式计算带来的提升约为12倍。函数的核心定义如下：

图 5: ts_regression_slope cython+流式计算代码

```
def class RollingRegression(Rolling):
    """Rolling regression with intercept"""
    cdef double x_sum
    cdef double y_sum
    cdef double x2_sum
    cdef double xy_sum
    cdef deque[double] x_havr

    def __init__(self, int window):
        super(RollingRegression, self).__init__(window)
        self.x_sum = 0
        self.y_sum = 0
        self.x2_sum = 0
        self.xy_sum = 0
        for i in range(window):
            self.x_havr.push_back(NAN)

    cdef tuple update2(self, double x_val, double y_val):
        self.x_havr.push_back(x_val)
        self.y_havr.push_back(y_val)

        cdef double x_front = self.x_havr.front()
        cdef double y_front = self.y_havr.front()

        if not isnan(x_front) and not isnan(y_front):
            self.x_sum += x_front
            self.y_sum += y_front
            self.x2_sum += x_front * x_front
            self.xy_sum += x_front * y_front
        else:
            self.na_count += 1

        self.x_havr.pop_front()
        self.y_havr.pop_front()

        if isnan(x_val) or isnan(y_val):
            self.na_count += 1
            # return NaN
        else:
            self.x_sum += x_val
            self.y_sum += y_val
            self.x2_sum += x_val * x_val
            self.xy_sum += x_val * y_val

        cdef int N = self.window - self.na_count

        cdef double numerator = N * self.xy_sum - self.x_sum * self.y_sum
        cdef double denominator = N * self.x2_sum - self.x_sum * self.x_sum
        cdef double slope = numerator / denominator
        cdef double intercept = (self.y_sum - slope * self.x_sum) / N

        return slope, intercept

    cdef double residual(self, double x_val, double y_val):
        cdef double slope, intercept
        slope, intercept = self.update2(x_val, y_val)
        return y_val - (slope * x_val + intercept)

    cdef double r_squared(self, double x_val, double y_val):
        cdef double slope, intercept
        slope, intercept = self.update2(x_val, y_val)
        cdef int N = self.window - self.na_count
        cdef double y_mean = self.y_sum / N
        cdef double ss_tot = 0.0
        cdef double ss_res = 0.0
        cdef double val
        cdef int i
        for i in range(self.y_havr.size()):
            val = self.y_havr[i]
            if not isnan(val):
                ss_tot += (val - y_mean) * (val - y_mean)
                ss_res += (val - (slope * self.x_havr[i] + intercept)) * (val - (slope * self.x_havr[i] + intercept))
        return 1 - (ss_res / ss_tot) if ss_tot != 0 else NaN
```

3.3 因子验证

因子验证模块主要用于计算因子的有效性或者适应度，评估因子有效性通常有两种方法，包括因子信息系数（Factor Information Coefficient, IC）和非线性特征重要性评估。

因子IC是一种计算因子预测能力的方法，它通过比较因子值与未来收益率之间的线性关系来衡量因子的预测能力。这种方法能够直观地反映出因子在预测市场表现方面的有效性。

非线性特征重要性评估则是一种更为复杂的方法，它考虑了因子在模型中的非线性作用，是机器学习用于理解模型决策过程和优化模型性能的关键技术，常用的算法包括基于树模型的特征重要性计算，SHAP值，排序重要性等，用于评估每个因子对模型整体性能的贡献。

对于基本面因子而言，我们更加注重因子的可解释性，因此本文主要采用因子IC来考察因子有效性，因子的收益率目标为第二天开盘价到一个月以后收盘价所对应的收益率。

3.4 因子筛选

因子筛选的主要目的是从批量生成的因子中筛选出效果较好的一些因子，个体筛选的算法有很多，常用的算法包括最优个体法，轮盘法以及锦标赛法。

最优个体法通过从因子种群中挑选出适应度得分最高的N个因子。这种方法的优势在于其高效性，能够迅速识别出表现最佳的因子。然而，它也存在一些局限性，特别是在进化算法中，这种方法可能会减少个体的多样性，从而增加陷入局部最优解的风险，同时也可能导致筛选出的因子之间存在过高的相关性。

轮盘法根据因子的适应度来确定其被选中的概率。这种方法有助于分散因子选择的集中度，但同时也存在一定的局限性，因为单个因子被选中的概率较低，这可能会降低最优因子被选中的可能性。此外，在进化算法中应用轮盘法可能会降低算法的进化速度。

锦标赛法是一种更为灵活的筛选方法。它首先在因子种群中随机选取k个个体，然后在这些因子中选择适应度最高的一个。这个过程重复进行，直到达到所需的个体数量。锦标赛法实际上是最优个体法和轮盘法的一般形式，通过调整k的值，可以在因子的适应度和群体多样性之间实现平衡。当k=1时，锦标赛法退化为轮盘法；当k等于种群大小时，锦标赛法等价于最优个体法。

本文中我们采用锦标赛法对候选个体进行筛选，其中参数k设定为3。这种方法虽然能在一定程度上缓解因子多样性的问题，但并不能从根本上解决这一问题。在进化算法中，随着迭代次数的增加，种群的整体适应度得以提升，然而，那些与现有因子相关性较低但自身适应度不高的因子，依然面临着难以被选入下一代的

风险。在先前的研究中，我们使用了退化变异和灾难算法，通过删除特定基因或个体来增强种群的多样性。在本研究中，我们采纳了一种更为直接和高效的策略：在适应度计算中引入了惩罚项。通过调整相关性惩罚后的适应度，我们能够筛选出既具有较高适应度又保持较低相关性的因子。适应度的计算公式定义如下：

$$fitness = IC * (1 - \mu * (1 - max_{corr}))$$

其中 μ 为惩罚系数，其取值范围从0到1， μ 值越大，对相关性的惩罚也就越重，这有助于提升种群的多样性，但同时可能会牺牲一定的整体有效性，本文取1/2。 max_{corr} 的计算方法如下：

$$max_{corr} = max(corr_{population}, corr_{factor\ pool})$$

max_{corr} 由两部分计算得到， $corr_{population}$ 表示个体与种群中其他个体的相关性， $corr_{factor\ pool}$ 表示个体与factor pool中的个体相关性，其中factor pool是已经经过筛选保留得到的因子池。

通过引入惩罚项，如果新生成的因子与现有因子的相关性过高，其适应度将被有效降低，从而减少其被选中的可能性，这有助于我们发现新的、具有潜力的因子

需要注意的是，如果直接计算因子间的截面相关性，其计算复杂度非常高，时间复杂度为 $O(MTP^2)$ ，其中T为时间长度，P为种群中个体数量，M为股票数量。为了提高计算效率，我们采用了因子IC时间序列的相关性来替代直接的因子值相关性计算，其计算效率为 $O(TP^2)$ ，与原始的因子值相关性计算相比，效率提升M（~5000）倍。

然而，因子值的相关性与因子IC的相关性之间可能会存在显著差异。尽管某些因子值本身可能不表现出强烈的相关性，但市场风格的趋同性可能导致因子IC的相关性显著高于因子值的相关性。以营收和净利润为例，尽管它们因子值的相关性为0.56，但它们的因子IC相关性却高达0.88。

从现实问题出发，我们更倾向于推荐使用IC相关性或因子收益率相关性的方法来评估因子之间的相关性。仅仅关注因子值的相关性可能只能确保我们的投资组合在表面上看起来是分散的，但这些投资标的可能在市场上表现出相似的行为，这在本质上并未能有效分散风险。相反，通过分析因子表现的相关性，我们可以从实际的市场表现角度出发，更有效地实现风险的分散。

3.5 因子进化

因子进化为可选步骤，此步骤专门用于优化和改进因子，对于进化算法而言，随机初始化生成的种群中不一定存在适应度高的个体，因此需要不断进化，使种群的适应度提高，从而筛选出效果更好的个体。

因子进化的原理主要是通过父代间的交叉和变异不断生成新的个体种群，再筛选出其中适应度较高的个体，从而使得整个种群的适应度不断增加，达到进化的目的。

在进化算法的应用中，将因子的字符串表达式表示为结构化表达式是至关重要的一步。通常，将因子解析为树状表达式是一种广泛采用的方法。除了树结构，还可以采用逆波兰表达式、程序表达式等其他形式。这三种表达方式各有千秋，能够相互转换，并且各自具有独特的优势：

- **逆波兰表达式**以其结构简洁著称，在进化过程中，通常通过添加或修改最后一个token来迭代生成新的因子。这种方法依赖于现有路径，因此搜索空间相对较小，但可以更精确地控制进化方向。
- **树状表达式**则提供了清晰的层次结构，在进化时可以在叶节点或算子上进行操作。与逆波兰表达式相比，树状表达式提供了更广阔的搜索空间，有助于探索更多可能的因子组合。
- **程序表达式**则提供了极高的灵活性，它允许在没有显式关联关系的代码片段之间进行操作。在进化过程中，如果不施加人为限制，可以在全搜索空间内进行探索。这种灵活性使得程序表达式在AutoML-Zero等框架中，被用来寻找全新的基础算法。然而，在全空间内进行搜索也意味着算法效率较低，在大规模搜索时需要一定的硬件资源

本文选择采用树状表达式来构建因子，树结构的优势在于，首先，能够清晰展示因子之间的关系和层次，使得结构易于理解 and 操作。其次，针对树结构的相关算法已经相当成熟，这为因子的进一步处理和优化提供了便利。此外，树结构的灵活性也使得算法能够更加高效地处理各种复杂的因子表达式

图 6:树状表达式

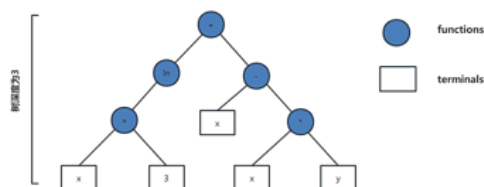


图 7:逆波兰表达式 (C)

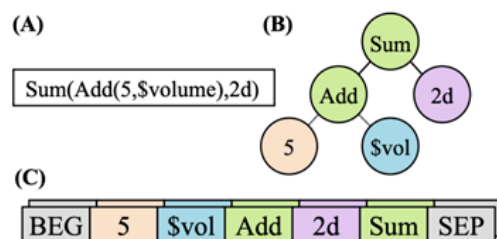


图 8:程序表达式

```
# sX/vX/mX = scalar/vector/matrix at address X.
# "gaussian" produces Gaussian IID random numbers.
def Setup():
    # Initialize variables.
    m1 = gaussian(-1e-10, 9e-09) # 1st layer weights
    s3 = 4.1 # Set learning rate
    v4 = gaussian(-0.033, 0.01) # 2nd layer weights
def Predict(): # v0=features
    v6 = dot(m1, v0) # Apply 1st layer weights
    v7 = maximum(0, v6) # Apply ReLU
    s1 = dot(v7, v4) # Compute prediction
def Learn(): # s0=label
    v3 = heaviside(v6, 1.0) # ReLU gradient
    s1 = s0 - s1 # Compute error
    s2 = s1 * s3 # Scale by learning rate
    v2 = s2 * v3 # Approx. 2nd layer weight delta
    v3 = v2 * v4 # Gradient w.r.t. activations
    m0 = outer(v3, v0) # 1st layer weight delta
    m1 = m1 + m0 # Update 1st layer weights
    v4 = v2 + v4 # Update 2nd layer weights
```

数据来源：AutoML-Zero: Evolving Machine Learning Algorithms From Scratch，中信建投证券

在进化过程中，父代个体通过子树交换和子树变异产生新的子代，两者的概率分别设置为0.6和0.3，以确保算法在探索新的可能性和保持已有优势之间取得平衡。在实际的进化过程中，我们不仅保留最终的hof（hall of fame）中的个体，而且对中间过程中产生的个体进行缓存。当计算新变异个体的适应度时，如果该个体已存在于缓存中，我们可以直接提取其适应度值，从而有效避免重复计算，有效提升运算效率。

在应用进化算法时，我们可以在初始因子生成阶段引入热启动策略。热启动的核心思想是将之前筛选得到的优异个体纳入初始化种群，这样做可以显著加速种群的进化进程，提高整体的搜索效率。通过这种方式，算法能够更快地逼近最优解。

4 实验设置

在本文中，我们采用的数据如前文所述，主要包括了三大财务报表，财务指标，一致预期以及市值等。对于这些基础数据，如果缺失值超过20%则予以剔除，最终共计201个字段，对每个字段都会赋予相应的量纲和频率，用于后续计算。数据属性样例如下：

表 3:数据属性

字段名	中文名	量纲	频率
tot_oper_rev	营业总收入	元	Q
s_fa_roe	净资产收益率	无	Q
val_mv	总市值	市值	D
est_total_profit_fy1	未来一年一致预期利润总额	元	D

资料来源：WIND，中信建投证券

数据为全市场股票从2014年至2024年5月底的所有数据，在计算因子有效性时，预测的目标为第2天的开盘价到21天后收盘价的收益率，每日滚动计算有效性，确保了因子的可交易性，避免路径依赖。

在因子生成时，主要采用随机化+热启动的方式，结合进化算法不断对因子进行改进。热启动因子的来源包括之前报告中构建的因子，以及在当前研究过程中发现的表现出色因子。在初始化阶段，我们采用"half-half"策略。具体来说，一半的初始个体是通过genFull方法生成的，这种方法确保了所有叶节点的深度与树的整体深度相等，从而生成了结构完整的因子。另一半个体则是通过genGrow方法生成的，这种方法允许至少有一个叶节点达到树的最大深度，同时其他叶节点可以有不同的深度，增加了因子的多样性。

在本研究中，我们设定种群大小为1000，意味着每一代种群中包含了1000个的因子。进化轮数设定为20，旨在避免因子结构变得过于复杂，确保模型的可解释性和实用性。在算法运行过程中，中间生成的因子，只要其信息系数（IC）超过0.04，就会被自动保存。算法运行结束后，将保存下来的优秀因子按照它们的IC值进行排序，并计算它们之间的两两相关性。如果两个因子之间的相关性超过0.6，我们会选择剔除IC值较低的因子，以此来构建候选因子池。

最后再将候选因子池与现有的因子池进行对比，计算它们之间的两两相关性。如果在候选因子中存在与现有因子池中因子相关性较低的新因子，这些新因子将被纳入现有的因子池。如果候选因子与现有因子池中的因子存在高相关性（超过0.6），并且候选因子的表现优于现有因子，则进行替换；否则保持现有因子池不变，不做任何调整。

5 结果

在对最终的因子经过严格的有效性和相关性筛选之后，我们得到了以下因子：

因子一： `ts_std(add(add(s_fa_yoyequity, s_qfa_optogr), s_qfa_yoysales), 12)`

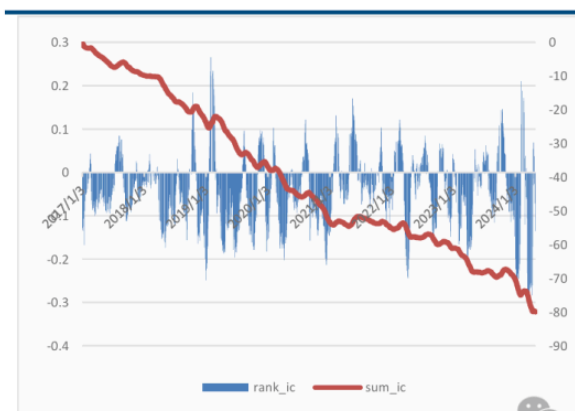
因子解释：12期波动率（相对年初增长率-归属母公司的股东权益（%）+单季度.营业利润 / 营业总收入 +单季度.营业收入同比增长率（%））

因子一主要表示公司增长和利润率的长期波动率，波动越小，未来收益越高。

因子IC： -0.0467

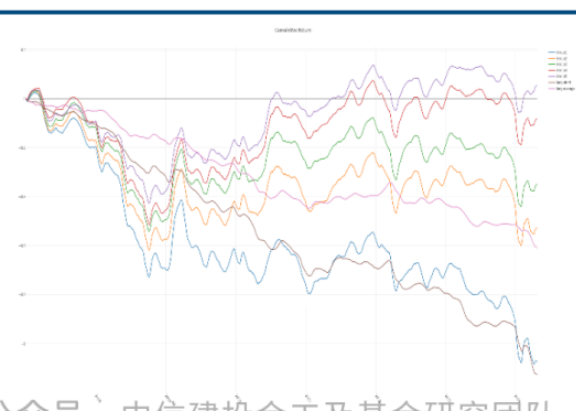
因子IR： 1.9615

图 9:因子一 IC



数据来源：WIND, 中信建投证券

图 10:因子一分组收益



数据来源：WIND, 中信建投证券

因子二：

$$\text{ts_regression_resi}(\text{add}(\text{net_cash_flows_oper_act}, \text{s_stm_bs}) / \text{ttm}(\text{s_fa_fcff}), \text{add}(\text{div}(\text{tot_shrhdr_eqy_excl_min_int}, \text{val_mv}), \text{qoq}(\text{add}(\text{tot_oper_cost2}, \text{oper_rev}))), 4)$$

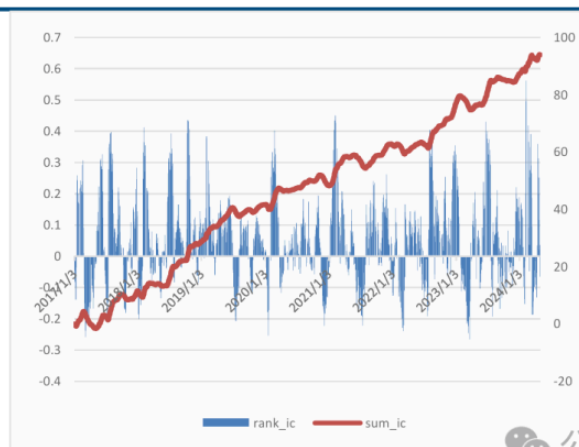
因子解释：4期回归残差（（经营活动产生的现金流量净额+固定资产合计）/TTM（企业自由现金流量（FCFF）），（股东权益合计（不含少数股东权益）/总市值 + 环比（营业总成本2+营业总收入））

因子二主要表示估值指标的可解释性，当第一项的增长高于第二项的增长时，未来收益越高。

因子IC：0.0567

因子IR：1.4513

图 11:因子二 IC



数据来源：WIND, 中信建投证券

图 12:因子二分组收益



数据来源：WIND, 中信建投证券

因子三：
$$\text{div}(\text{ts_mean}(\text{surplus_rsrv}, 8), \text{val_mv})$$

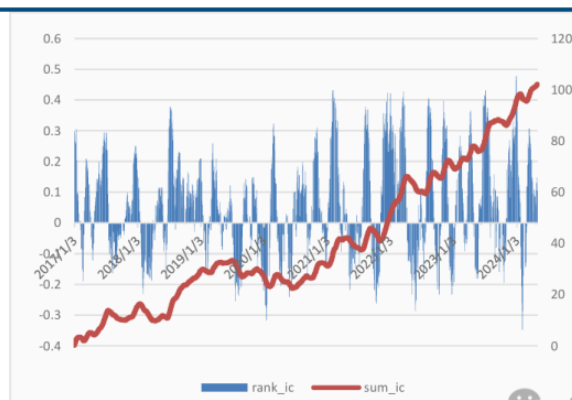
因子解释：8期平均（盈余公积金）/总市值

因子三是类PE的估值因子，采用了盈余公积金取代了PE中的净利润，该因子值越大，未来收益越高。

因子IC：0.0537

因子IR：1.2734

图 13:因子三 IC



数据来源: WIND, 中信建投证券

图 14:因子三分组收益



数据来源: WIND, 中信建投证券

因子四: $\text{div}(\text{ts_mean}(\text{surplus_rsrv}, 8), \text{val_mv})$

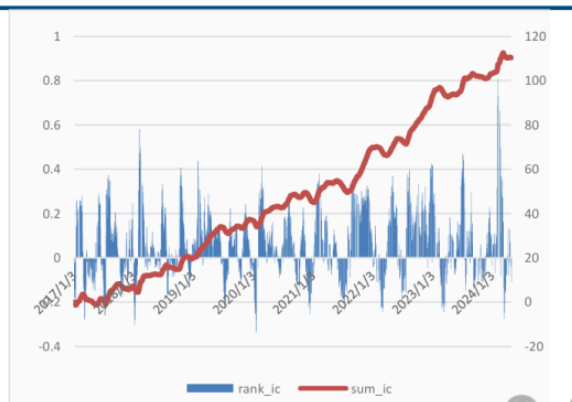
因子解释: 2个月变化率 (盈余公积金/总市值)

因子四是估值因子的变化率, 估值因子结构与因子三类似, 变化主要来自两方面, 一是相对低频的盈余公积金变化, 二是高频变动的市值。

因子IC: 0.0720

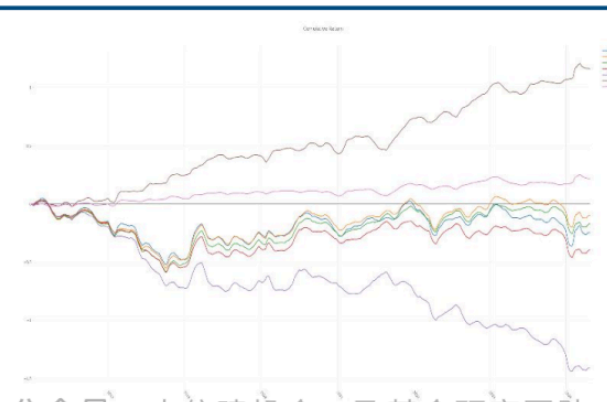
因子IR: 1.6311

图 15:因子四 IC



数据来源: WIND, 中信建投证券

图 16:因子四分组收益



数据来源: WIND, 中信建投证券

6 总结

在本篇研究中, 我们在先前系列因子挖掘工作的基础上, 精心构建了一个全面的基本面因子挖掘统一框架, 该框架不仅整合了多种因子生成技术, 并且能够自动优化基本面因子的频率和量纲。在此框架中融入了一系列常用算子, 包括元素算子、时序算子和截面算子等, 此外, 为了进一步提升计算效率, 我们在框架的底层采用了cython与流式计算技术的结合, 这显著提高了因子计算的速度和性能。在因子筛选阶段, 我们特别引入了因子相关性惩罚机制。这一机制的运用有效促进了因子的多样性, 确保了最

终生成的因子集合在保持低相关性的同时，也能捕捉到市场的有效信息。最终，我们展示了一些相关性较低且表现出色的基本面因子

风险提示

本报告中所有数据结果是基于历史统计结果的展示，未来有可能发生风格切换导致因子失效的风险。模型运行存在一定的随机性，初始化随机数种子会对结果产生影响，单次运行结果可能会有一定偏差。历史数据的区间选择会对结果产生一定的影响。模型参数的不同会影响最终结果。模型对计算资源要求较高，运算量不足会导致结果存在一定的欠拟合风险。本文所有模型结果均来自历史数据，模型存在统计误差，不保证模型未来的有效性，对投资不构成任何建议。

分析师介绍

姚紫薇，金融工程及基金研究首席分析师。上海财经大学管理学硕士，厦门大学统计学学士，在基金研究、资产配置、产品设计、财富管理等领域均有长期深入研究。曾担任招商证券基金评价业务负责人，多次获得“新财富”金融工程方向前三（团队核心成员）。
王超，南京大学粒子物理博士，曾担任基金公司研究员，券商研究员，有丰富的研究和投资经验，2021年加入中信建投证券研究所，主要负责量化多因子选股。

证券研究报告名称：《“逐鹿”Alpha 专题报告（二十二）——Factor Zoo II 基本面因子挖掘统一框架》
对外发布时间：2024年8月7日
报告发布机构 中信建投证券股份有限公司
本报告分析师：
姚紫薇
SAC编号：S1440524040001
王超
SAC编号：S1440522120002

免责声明：

【中信建投金工及基金研究团队】
本订阅号（微信号：中信建投金工及基金研究团队）为中信建投证券股份有限公司（下称“中信建投”）研究发展部金工及基金研究团队运营的唯一订阅号。
本订阅号所载内容仅面向符合《证券期货投资者适当性管理办法》规定的机构类专业投资者。中信建投不因任何订阅或接收本订阅号内容的行为而将订阅人视为中信建投的客户。
本订阅号不是中信建投研究报告的发布平台，所载内容均来自于中信建投已正式发布的研究报告或对报告进行的跟踪与解读，订阅者若使用所载资料，有可能会因缺乏对完整报告的了解而对其中关键假设、评级、目标价等内容产生误解。提请订阅者参阅由中信建投已发布的完整证券研究报告，仔细阅读其所附

专题研究 · 目录

上一篇

“逐鹿”Alpha 专题报告（二十二）——Factor Zoo II 基本面因子挖掘统一框架

下一篇

股票关联与溢出效应因子构建

